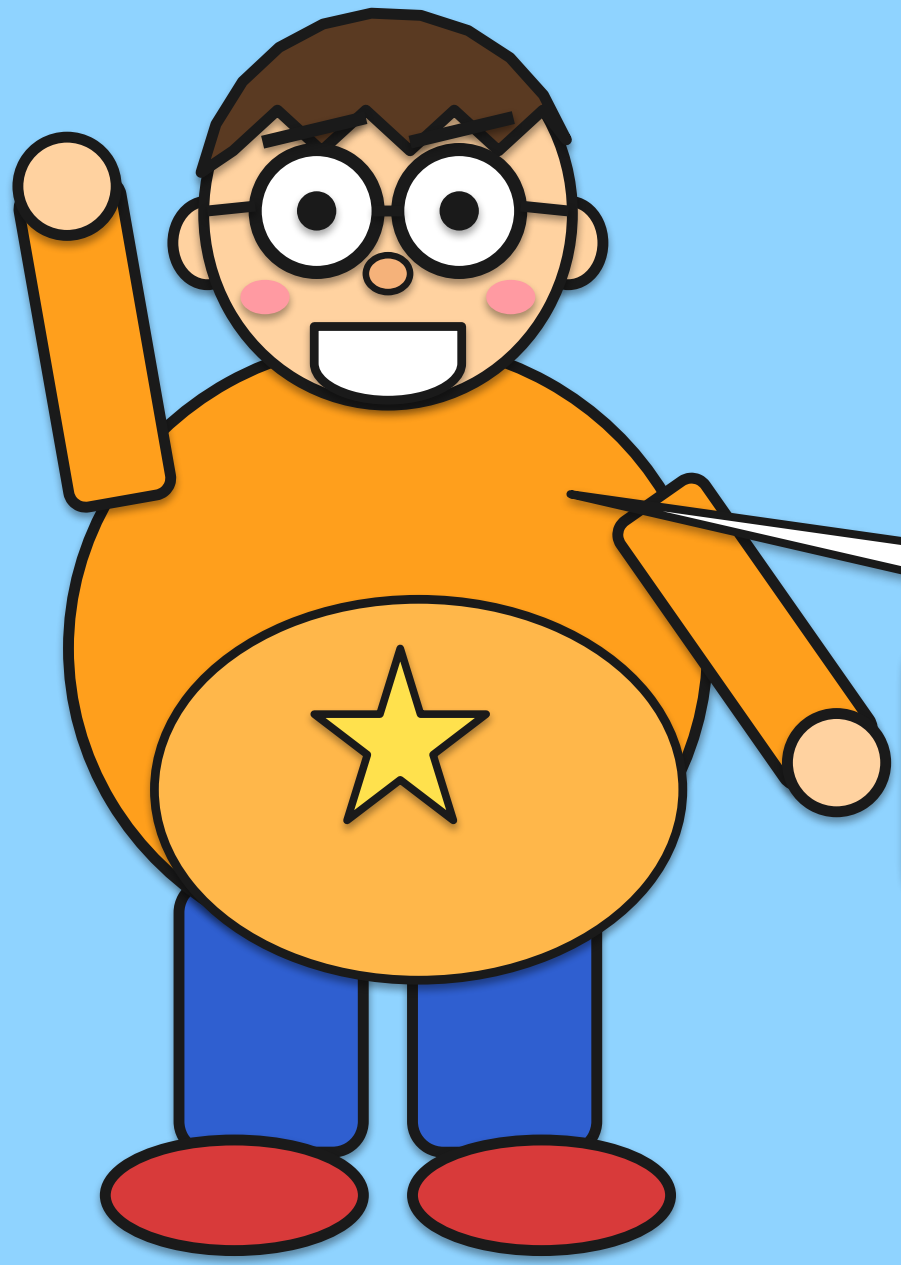
JUMP & RUN

in MakeCode Arcade – mit Blöcken!

Sprung · Plattformen · Level · Coins · Gegner-KI · Boss

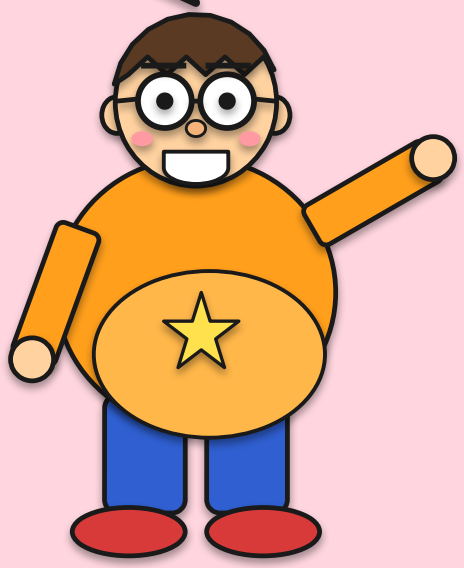


Hallo! Ich bin Paule Pixel! Heute lernt ihr, wie man hüpft, Münzen sammelt und Bosse verhaut!

Los!

Unser Fahrplan – 10 Comic-Panels bis zum Boss

Wir bauen Stück für Stück ein komplettes Spiel. Jede Folie = ein Baustein, den du direkt in MakeCode nachbauen kannst. Los geht's!

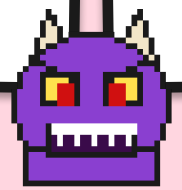


1. **Projekt, Kacheln & Map**
2. **Laufen & Schwerkraft**
3. **Einfach- & Doppelsprung**
4. **Animationen**
5. **Level per Funktion**
6. **Animierte Coins**
7. **Gegner mit KI**
8. **Bosskampf in Phasen**
9. **Gesamtbild**
10. **Tipps & Challenges**





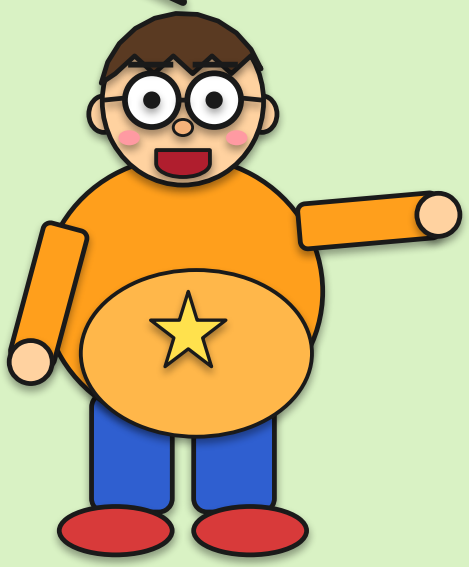




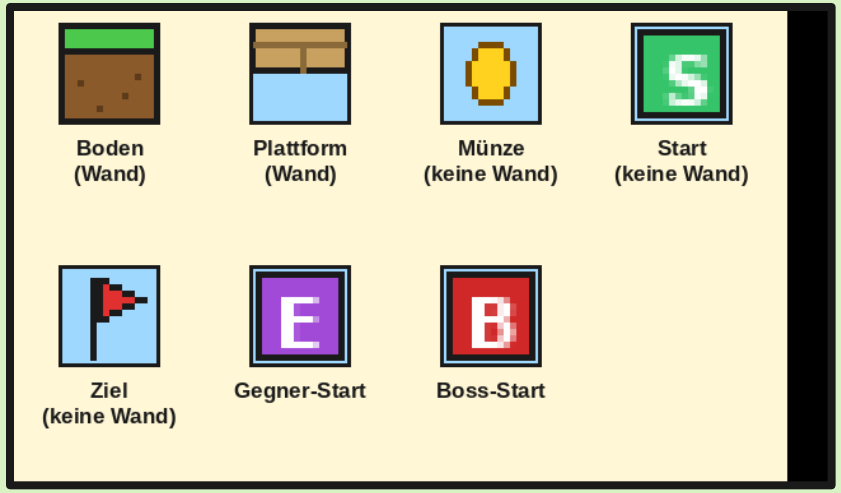
Blöcke: Farben angelehnt an MakeCode

1 Projekt anlegen & Kacheln (Tiles) vorbereiten

Erst das Fundament: Ohne Kacheln keine Map! Wichtig: Boden & Plattformen bekommen im Kachel-Editor die Wand-Markierung. Alles andere sind nur Platzhalter!



1. **makecode.com/arcade ▸ Neues Projekt „JumpAndRun“**
2. **Sprites ▸ SpriteKind: Player, Enemy, Food (=Münze) + neu: Boss**
3. **Szene ▸ Tilemap ▸ z. B. 32×16 Kacheln (16 px)**
4. **Kacheln malen; nur Boden/Plattform als „Wand“ markieren**
5. **Platzhalter-Kacheln: Start, Ziel, Münze, Gegner-Start, Boss-Start**

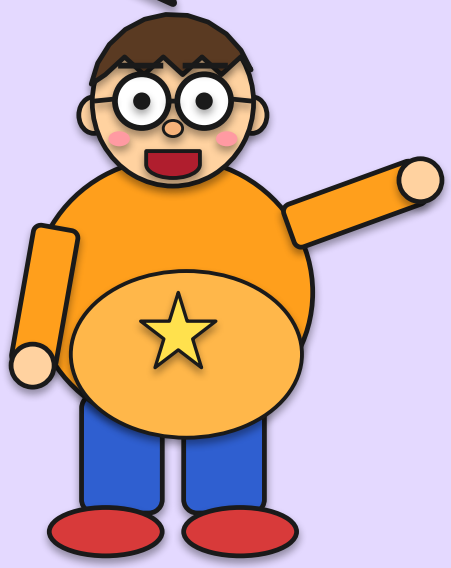


Merke:
Wand = Spieler und Gegner können nicht hindurch. Ziel, Münze & Co. sind KEINE Wände – wir ersetzen sie beim Levelstart durch Sprites oder leere Kacheln.

2

Spieler: Laufen & Schwerkraft

Der Trick der Schwerkraft: eine ständige Beschleunigung nach unten (ay). Bei den Tasten geben wir nur die Seitwärts-Geschwindigkeit frei – sonst fliegt Paule!



Beim Start

beim Start

- setze player auf Sprite [Paule 16×16] der Art Player
- bewege player mit Tasten vx 100 vy 0
- setze player Beschleunigung ay auf 300
- Kamera folgt player
- setze Leben auf 3





ay = 300 ⇒ fällt

Achtung:
vy NICHT an die Tasten binden (0 lassen), sonst steuerst du hoch/runter.
ay größer = schwerer, kleiner = Mond-Gefühl.

Faustregel:
Pixelmaß: 16 px = 1 Kachel.
Tempo 100 px/s ≈ 6 Kacheln pro Sekunde.

3a

Der Einfachsprung

Sprung = kurzer Stoß nach oben: vy auf −150. Aber nur, wenn Paule auf dem Boden steht – dafür fragen wir die Kollision nach unten ab!



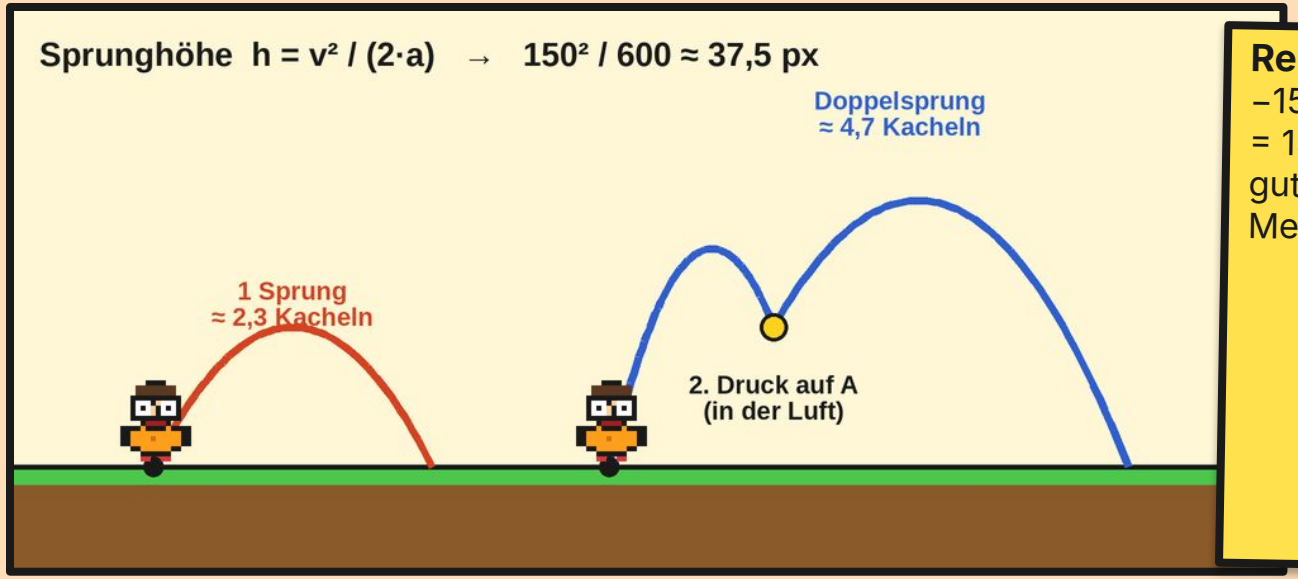
Sprung auslösen

wenn Taste A gedrückt

wenn <player berührt Wand [unten]> dann

setze player Geschwindigkeit vy auf -150

Kategorien:
Steuerung ▸ A-Taste
Logik ▸ wenn
Sprites ▸ berührt Wand

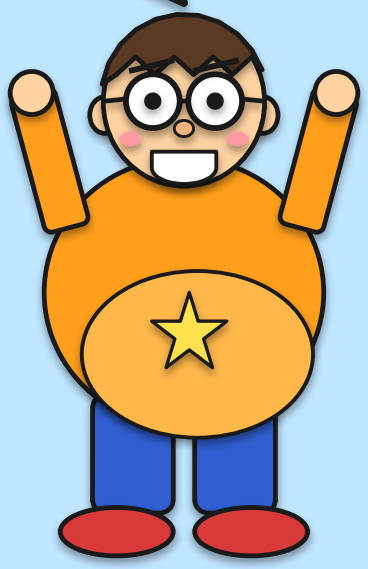


Rechnung:
−150 px/s und ay = 300 ergibt h = 150² / (2·300) ≈ 37 px, also gut 2 Kacheln.
Mehr Höhe? vy ↑ (z. B. −180).

3b

Der Doppelsprung

Wir zählen die Sprünge! Erlaubt sind 2. Auf dem Boden setzen wir den Zähler zurück – aber nur, wenn Paule nicht gerade nach oben fliegt (vy ≥ 0)!



Variable: Sprünge

```
beim Start
    setze [Sprünge] auf 0
```

A-Taste

```
wenn Taste A gedrückt
    wenn  Sprünge < 2  dann
        setze player  vy  auf -150
        ändere [Sprünge] um 1
```

Zurücksetzen am Boden

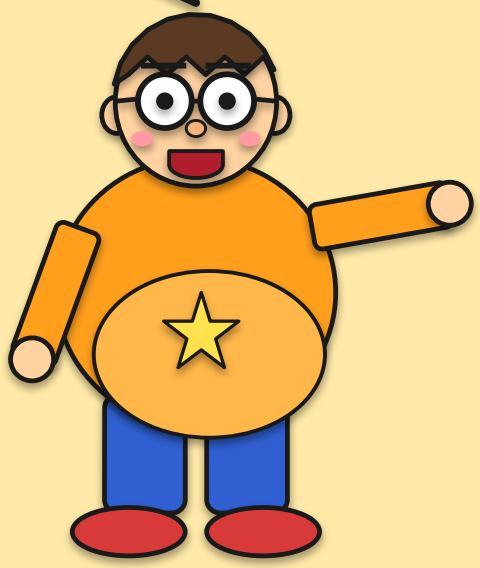
```
bei jedem Spiel-Update
    wenn <player berührt Wand [unten]  und
        player vy ≥ 0>  dann
            setze [Sprünge] auf 0
```

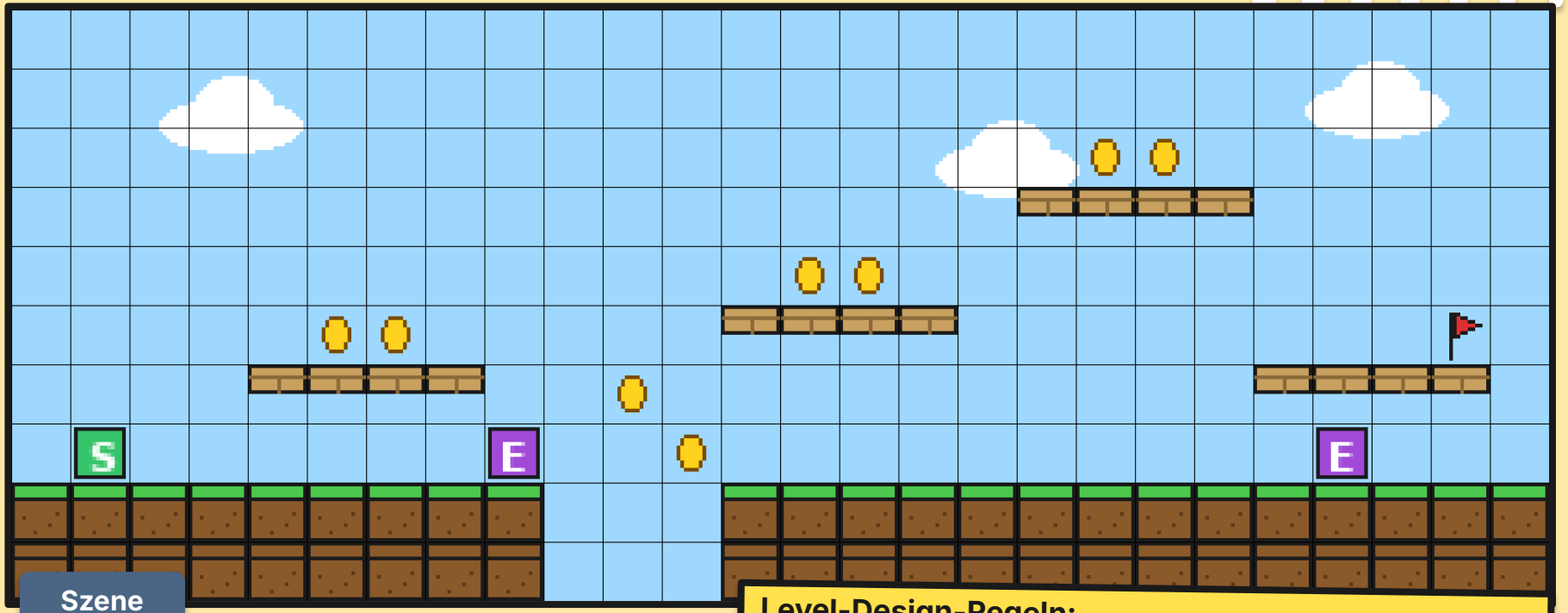
Bug-Falle:
Ohne „vy ≥ 0“ wäre der Zähler im Sprungmoment noch am Boden → unendlich viele Sprünge!
Zahl 2 → 3 = Dreifachsprung.

4

Plattformen, Map & Kamera

Malt die Welt wie ein Level-Designer: Boden, Plattformen, Lücken. Die Platzhalter (S, Ziel, Münzen, E) setzt du einfach auf die Karte!





Szene

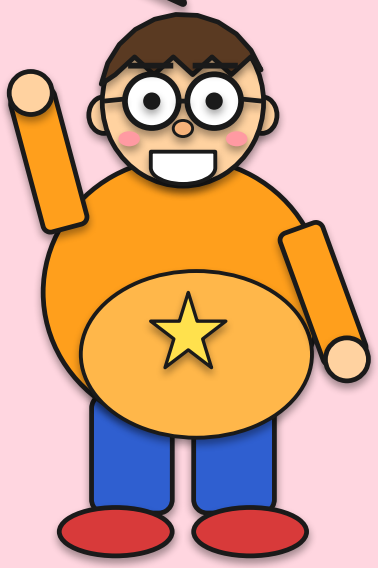
- setze Hintergrundfarbe auf [hellblau]
- setze Tilemap auf [level1]
- Kamera folgt player

Level-Design-Regeln:
Sprunghöhe ≈ 2,3 Kacheln → Plattformen höchstens 2 Kacheln höher bauen (Doppelsprung: bis 4).
Lücken ≤ 5 Kacheln breit.
Dunkle Linien = Raster der Kacheln.

5

Animationen: Laufen, Stehen, Springen

Mit der Erweiterung „character animations“ (Erweiterungen ▸ suchen) wählt Paule automatisch den passenden Film – je nach Bewegung!





Stehen



Lauf 1



Lauf 2



Sprung

Tipp:
Links-Bilder: im Bildeditor „spiegeln“ (⇆).
Regeln mit mehr Bedingungen (z. B. links + oben) gewinnen.

Regeln (beim Start)

```
player: Bilder in Schleife [Lauf rechts] alle 100 ms  wenn [bewegt nach rechts]
player: Bilder in Schleife [Lauf links] alle 100 ms  wenn [bewegt nach links]
player: Bilder in Schleife [Stehen] alle 100 ms  wenn [steht still]
player: Bilder in Schleife [Sprung] alle 100 ms  wenn [bewegt nach oben]
```

6

Mehrere Level – per Funktionsaufruf

Alles, was beim Levelstart passiert, packen wir in EINE Funktion: starteLevel(nr). Ziel erreicht? Level +1 und Funktion nochmal aufrufen. Fertig!



Funktion starteLevel (nr)

```
Funktion  starteLevel (nr)
    für [s] aus Sprites der Art Food: zerstöre s
    für [s] aus Sprites der Art Enemy: zerstöre s
    wenn  nr = 1  dann
        setze Tilemap auf [level1]
    sonst wenn  nr = 2  dann
        setze Tilemap auf [level2]
    sonst
        setze Tilemap auf [level3]
    für Element [ort] aus alle Orte der Kachel [Start]
        platziere player auf ort
        setze Kachel bei ort auf [leer]
    rufe  erzeugeMünzen
    rufe  erzeugeGegner
```

Start

```
beim Start
    setze [Level] auf 1
    rufe  starteLevel  (Level)
```

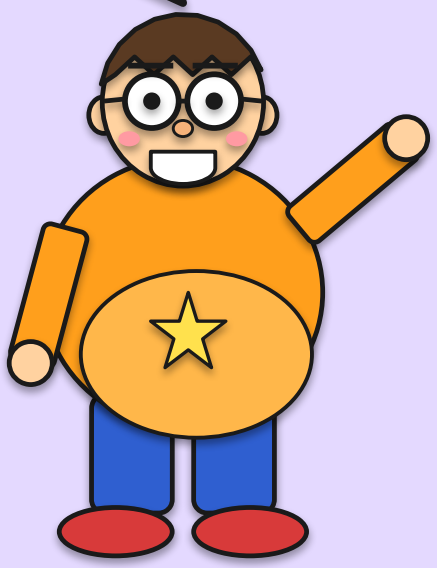
Ziel erreicht

```
wenn Player Kachel [Ziel] berührt
    ändere [Level] um 1
    wenn  Level > 3  dann
        Spiel vorbei – gewonnen
    sonst
        rufe  starteLevel  (Level)
```

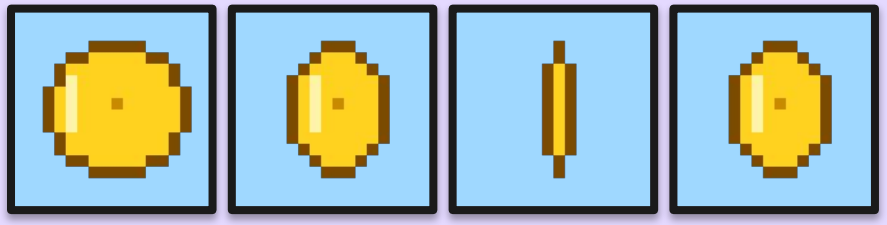
7

Animierte Coins: Kachel → bewegter Sprite

Der Clou: Du malst nur einfache Münz-Kacheln in die Map. Beim Levelstart ersetzt eine For-Schleife jede Kachel durch eine drehende Münze!

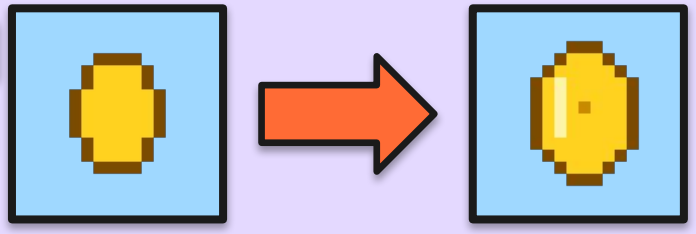


```
Funktion erzeugeMünzen
Funktion  erzeugeMünzen
    für Element [ort] aus alle Orte der Kachel [Münze]
        setze [coin] auf Sprite [Münze 1] der Art Food
        platziere coin auf ort
        animiere coin: [Münze 1–4] 120 ms, Schleife
        setze Kachel bei ort auf [leer]
```



Frame 1 Frame 2 Frame 3 Frame 4

So läuft's ab:



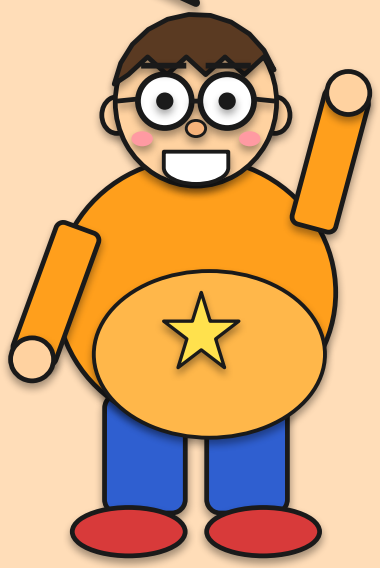
Kachel (Map) Sprite (dreht sich)

Muster merken:
Erst Sprite erzeugen, DANN die Kachel durch „leer“ ersetzen – sonst bleibt die gemalte Münze stehen.
Wie bei Start & Gegnern: gleiches Muster!

7b

Münzen einsammeln, Punkte & Leben

Überlappen Spieler und Münze → Punkt, Funken, Sound, Münze weg! Und wer in ein Loch fällt, verliert ein Leben.



Münze nehmen

```
wenn Player berührt Food
    ändere Punktestand um 1
    zerstöre otherSprite Effekt [Sprühen] 100 ms
    spiele Ton [ba ding]
```

In den Abgrund gefallen

```
bei jedem Spiel-Update
    wenn player y > 320 dann
        ändere Leben um -1
        rufe starteLevel (Level)
```

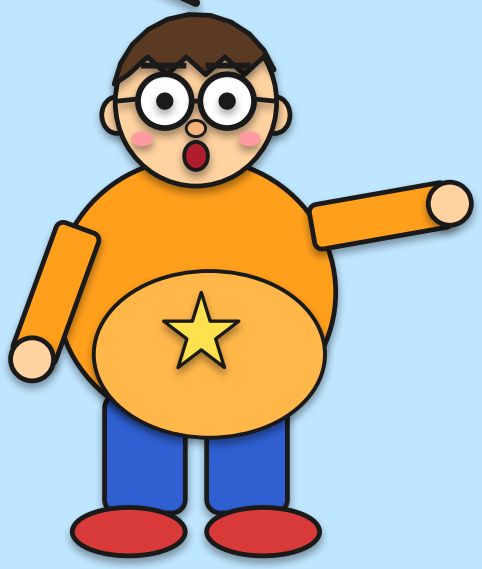
Gratis dabei:
Punkte & Leben zeigt MakeCode automatisch oben im Bild an.
Bei 0 Leben endet das Spiel von allein (Game Over).

Idee:
Bonus: alle 10 Münzen ein Extra-Leben:
wenn Punkte mod 10 = 0 → Leben +1

8a

Gegner-KI Stufe 1: Patrouille

Ein guter Gegner läuft hin und her, dreht an Wänden um – und stürzt nicht von Kanten! Dazu schaut er eine Kachel nach vorne und unten.



Gegner erzeugen (Funktion erzeugeGegner)

```
Funktion  erzeugeGegner
    für Element [ort] aus alle Orte der Kachel [Gegner-Start]
        setze [gegner] auf Sprite [Schleim] der Art Enemy
        platziere gegner auf ort;  gegner ay = 300
        setze Daten [dir] von gegner auf 1
        setze Kachel bei ort auf [leer]
```

Patrouillen-Logik

```
bei jedem Spiel-Update
    für Element [g] aus alle Sprites der Art Enemy
        wenn  g berührt Wand [links] oder [rechts]  oder
              nicht [Kachel vorn-unten ist Wand]  dann
                  setze [dir] von g auf  –dir
        setze g vx auf  dir × 30
```



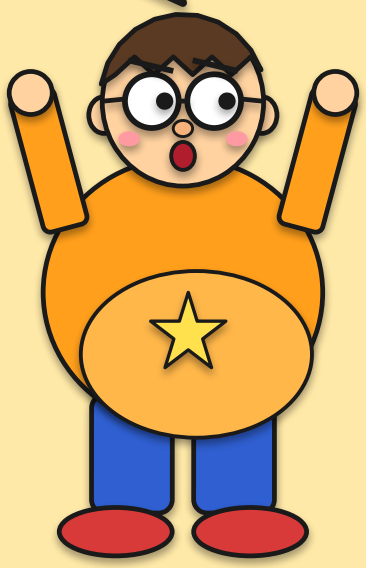
← Schleim patrouilliert →

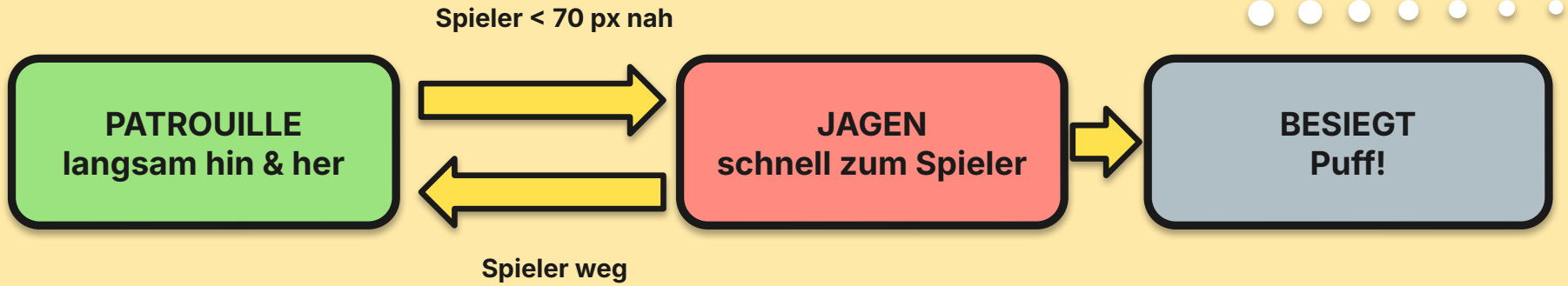
Kante erkennen:
Kachel „vorn-unten“ = Spalte + Blickrichtung, Zeile + 1.
Blöcke: Szene ▸ Kachelposition (Spalte, Zeile) und „ist Wand“.
Richtung als Daten speichern, weil eine Wandberührung vx auf 0 setzt!

8b

Gegner-KI Stufe 2: Sehen & Jagen

Jetzt wird's gemein: Sieht der Gegner Paule in Reichweite, jagt er ihn! Das ist ein einfacher Zustandsautomat – genau so tickt echte Spiel-KI.





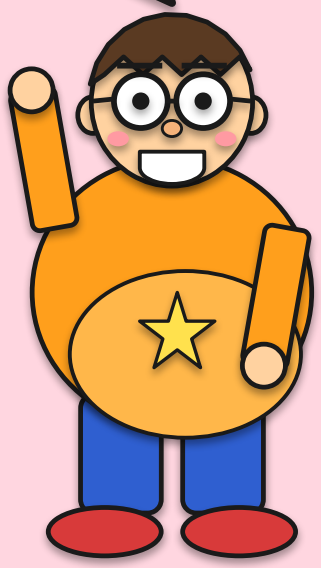

```
Entscheidung pro Gegner
bei jedem Spiel-Update
    für Element [g] aus alle Sprites der Art Enemy
        wenn  |player x – g x| < 70  und  |player y – g y| < 24  dann
            wenn  player x > g x  dann  vx = 45   sonst  vx = -45
        sonst
            Patrouille (Kante & Wand wie vorige Folie)
```

Variieren:
Unterschiedliche „Sichtweiten“ (70, 100, 40) → Gegner mit eigener Persönlichkeit.
Fliegender Gegner? ay = 0 und auch vy zum Spieler setzen.

8c

Gegner besiegen: Von oben draufhüpfen!

Trifft Paule von oben (er fällt und seine Füße sind über dem Gegner) → Gegner weg + Bounce! Sonst tut's weh – dann kurz unverwundbar.

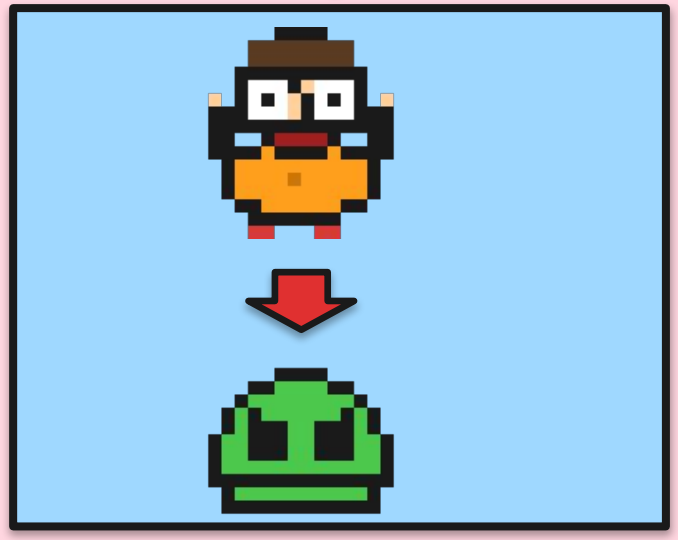


Funktion hurtPlayer

```
Funktion hurtPlayer
    wenn nicht [unverwundbar] dann
        setze [unverwundbar] auf wahr
        ändere Leben um -1; player vy = -120
        im Hintergrund ausführen:
            pause 1000 ms; setze [unverwundbar] auf falsch
```

Spieler trifft Gegner

```
wenn Player berührt Enemy
    wenn player vy > 0 und player unten < otherSprite y dann
        zerstöre otherSprite; player vy = -120; Punkte +5
    sonst
        rufe hurtPlayer
```



Fällt von oben = Stomp ✔️

Wiederverwenden:
Dieselbe Funktion hurtPlayer nutzen wir später für Boss & Feuerbälle!

9a

Bosskampf: Phasen statt Dauerfeuer

Ein guter Boss ändert sein Verhalten, wenn er Leben verliert. Wir bauen 3 Phasen – und jede gibt dem Boss neue, gemeine Tricks!



Boss-Leben



10 → 6 Leben | 5 → 3 | 2 → 0



Phase 1

Ruhiger Start

- läuft auf Spieler zu
- gelegentlich Sprung

Phase 2

Wird wütend

- schneller
- + wirft Feuerbälle

Phase 3

Raserei!

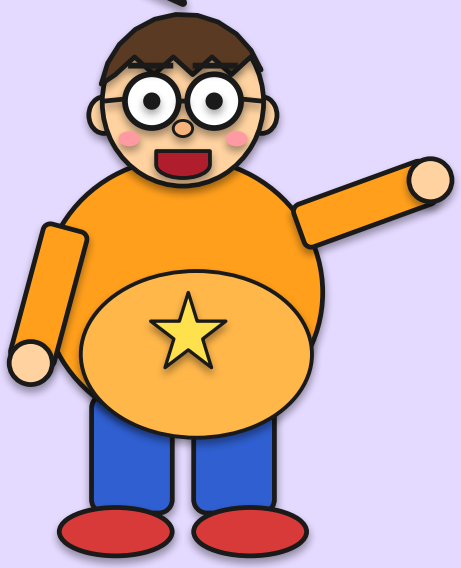
- noch schneller
- Sprung + Feuerball
- kurze Pausen

Intelligenz heißt:
Faires Design: Vor jedem Angriff kurz blinken lassen (Telegraph) – so kann der Spieler reagieren. Boss nur von oben verwundbar, 0,8 s Schonzeit nach jedem Treffer.

9b

Boss bauen: Leben, Balken & Treffer

Boss erzeugen wir wie die Gegner: Platzhalter-Kachel suchen, Sprite platzieren. Dazu ein Lebensbalken über dem Kopf (Erweiterung „status bar“).



Boss erzeugen

```
Funktion erzeugeBoss
    für Element [ort] aus alle Orte der Kachel [Boss-Start]
        setze [boss] auf Sprite [Boss 32×32] der Art Boss; platziere auf ort
        setze boss ay auf 300; Kachel bei ort auf [leer]
    setze [BossLeben] auf 10; setze [Phase] auf 1
    setze [bossBar] auf Statusbalken 24×4 Art [Gegnerleben]
    hänge bossBar an boss; Max = 10; Wert = 10
```

Treffer auf den Boss

```
wenn Player berührt Boss
    wenn player vy > 0 und unten < Boss oben + 8
        wenn nicht [bossHurt] dann
            bossHurt = wahr; BossLeben −1; Balken aktualisieren
            Phase setzen (≤5→2, ≤2→3); bei 0: Sieg!
    sonst: rufe hurtPlayer
```

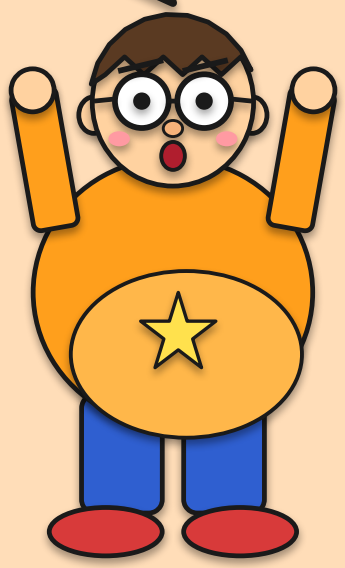


Wichtig:
„Sieg“ = Spiel vorbei – gewonnen, oder: Tür/Ziel-Kachel erscheinen lassen.
bossHurt-Flag verhindert, dass ein Sprung mehrere Treffer zählt.

9c

Boss-KI: Angriffe per Zufall & Phase

Alle 1,5 Sekunden entscheidet der Boss neu: Anlauf, Sprung oder Feuerball. Je höher die Phase, desto schneller – und der Feuerball wird erst in Phase 2 freigeschaltet!



Angriffs-Auswahl

```
alle 1500 ms
    wenn BossLeben > 0 dann
        setze [Richtung] auf 1 wenn player rechts, sonst -1
        setze [Angriff] auf Zufallszahl von 0 bis 2
        wenn Angriff = 0 dann
            setze boss vx auf Richtung × (40 + 25 × Phase)
        sonst wenn Angriff = 1 und boss am Boden
            setze boss vy auf -160; boss vx auf Richtung × 50
        sonst wenn Phase ≥ 2 dann
            boss: Effekt [warmes Glühen] 400 ms; pause 400 ms
            Feuerball: vx = Richtung × (70 + 20 × Phase)
```

Feuerball trifft

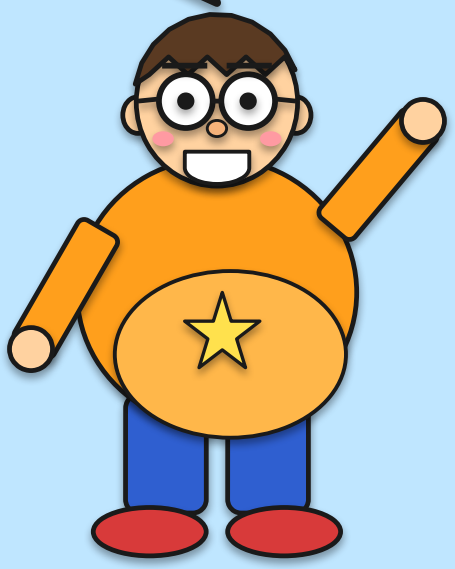
```
wenn Player berührt Projektil
    zerstöre otherSprite
    rufe hurtPlayer
```

Feinschliff:
Blinken vor dem Schuss (400 ms) = Warnung für den Spieler.
Ohne Wände in der Arena? Dann prallt der Boss mit vx = −vx an den Rändern ab.
Mehr Moves: Schockwelle beim Landen, Minions rufen …

10

Das Gesamtbild: Wer ruft wen?

So fügt sich alles zusammen: Der Start ruft starteLevel, die Funktion erzeugt Münzen, Gegner und – im letzten Level – den Boss. Events laufen ständig mit!



beim Start → **starteLevel (nr)**

- **erzeugeMünzen**
- **erzeugeGegner**
- **erzeugeBoss (nur letztes Level)**

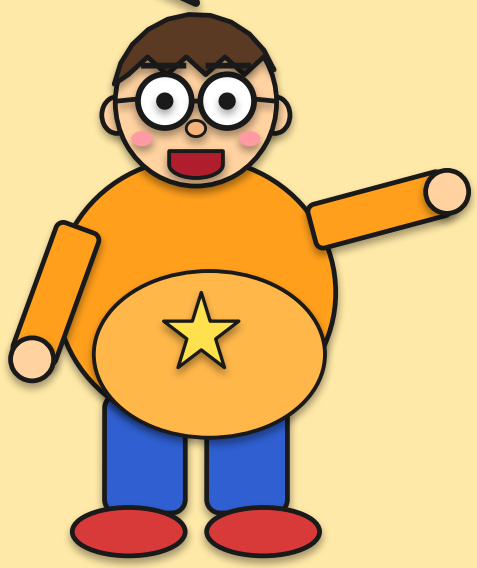
Events, die ständig lauschen:

A-Taste	Update	Überlappung	Ziel-Kachel	Intervall 1,5 s
Einfach-/ Doppelsprung	Boden-Reset, Sturz, Gegner-KI	Münze · Gegner · Boss · Feuerball	Level +1	Boss-Angriffe

Tipp: Jede Funktion klein halten – testen, dann weiter!

Typische Fehler – und die Lösung

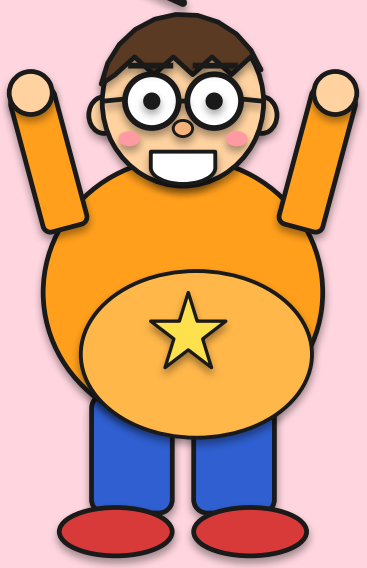
Keine Panik! Fast jeder Bug im Jump & Run kommt aus dieser Liste. Erst hier nachschauen, dann verzweifeln. 😄



Paule fällt durch den Boden	**Kachel im Editor als „Wand“ markieren**
Unendlich viele Sprünge	**Reset nur bei Boden UND vy ≥ 0 (Folie 3b)**
Münzen/Gegner bleiben im neuen Level	**Am Funktionsanfang alte Sprites zerstören**
Gegner stehen still an der Wand	**Richtung in Daten speichern, nicht aus vx lesen**
Mehrere Leben auf einmal weg	**Unverwundbar-Flag + Pause im Hintergrund**
Level startet doppelt	**Kachel-Treffer nur einmal: Flag setzen oder Spieler kurz „Ghost“**
Spiel ruckelt	**Weniger Sprites (< 100); Münzen außerhalb der Kamera nicht animieren**

Deine Challenges – mach das Spiel zu deinem!

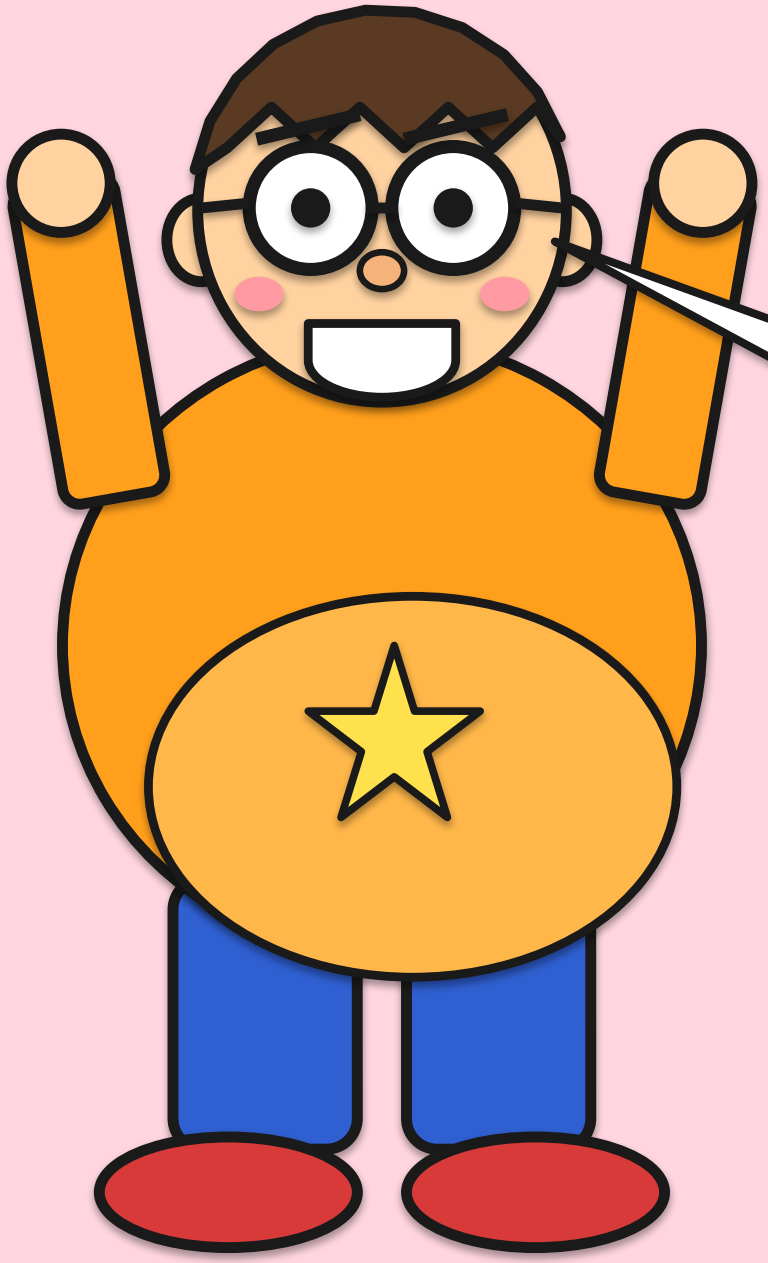
Jetzt bist du dran! Such dir 1–2 Aufgaben aus, baue sie ein und zeig sie deinen Freunden. Sterne zeigen die Schwierigkeit.



★ Münzen schweben leicht auf und ab	★ Checkpoint-Kachel: neu starten ab Fahne
★★ Coyote Time: kurz nach der Kante noch springen	★★ Power-Up: Pilz = 10 s Dreifachsprung
★★ Schlüssel sammeln, Tür öffnet das Ziel	★★★ Wandsprung wie bei Mega Man
★★★ Bewegliche Plattform, auf der man mitfährt	★★★ Zweiter Boss mit eigener Phase 4

Alle Bausteine stehen in den Folien-Notizen auch als JavaScript-Code.

VIEL SPASS BEIM HÜPFEN!



Und denk dran: Fehler sind nur Level, die du noch nicht geschafft hast. Bis zum nächsten Boss!

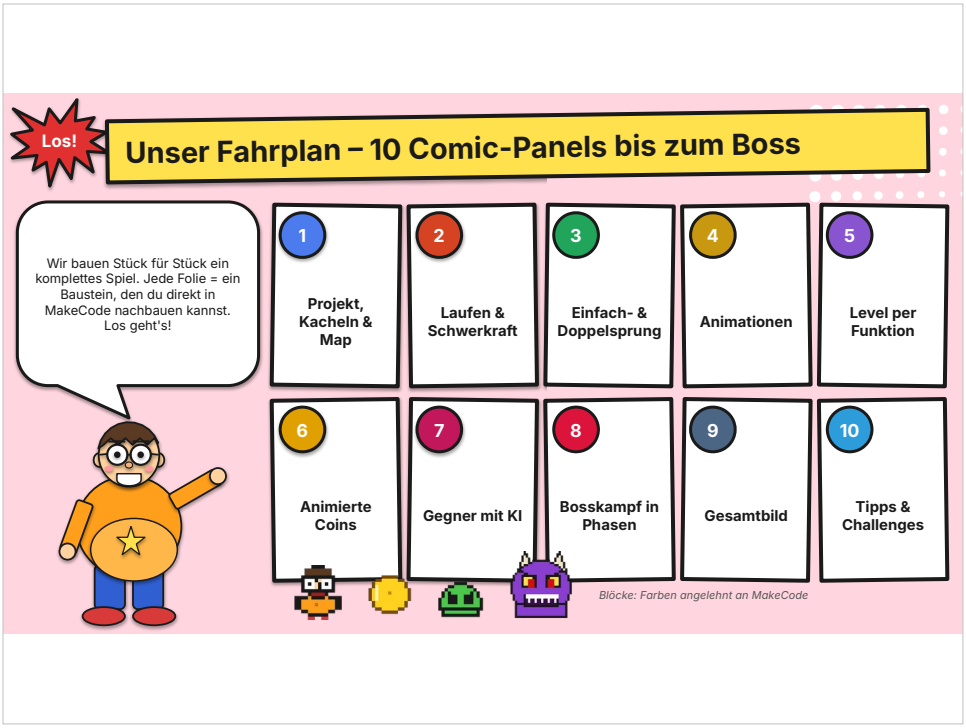








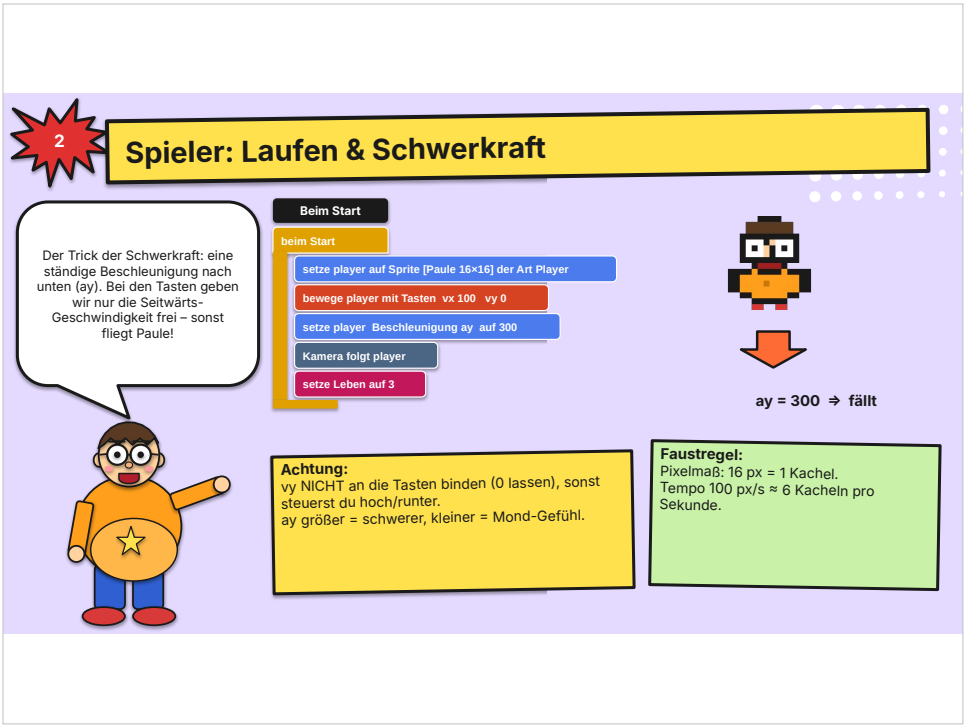
Titelfolie. Paule Pixel führt durch die Anleitung. Alle Blöcke sind sinngemäß in deutscher MakeCode-Arcade-Oberfläche beschrieben; in den Notizen jeder Folie steht der passende JavaScript-Code (im Editor oben auf 'JavaScript' umschalten).



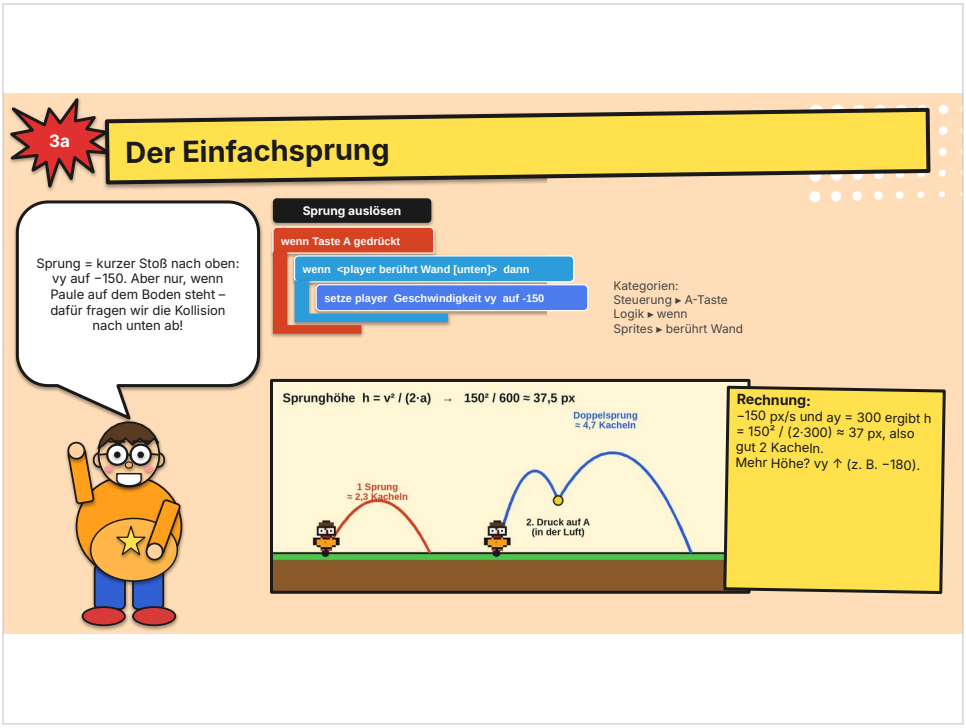
Übersicht über alle Kapitel.



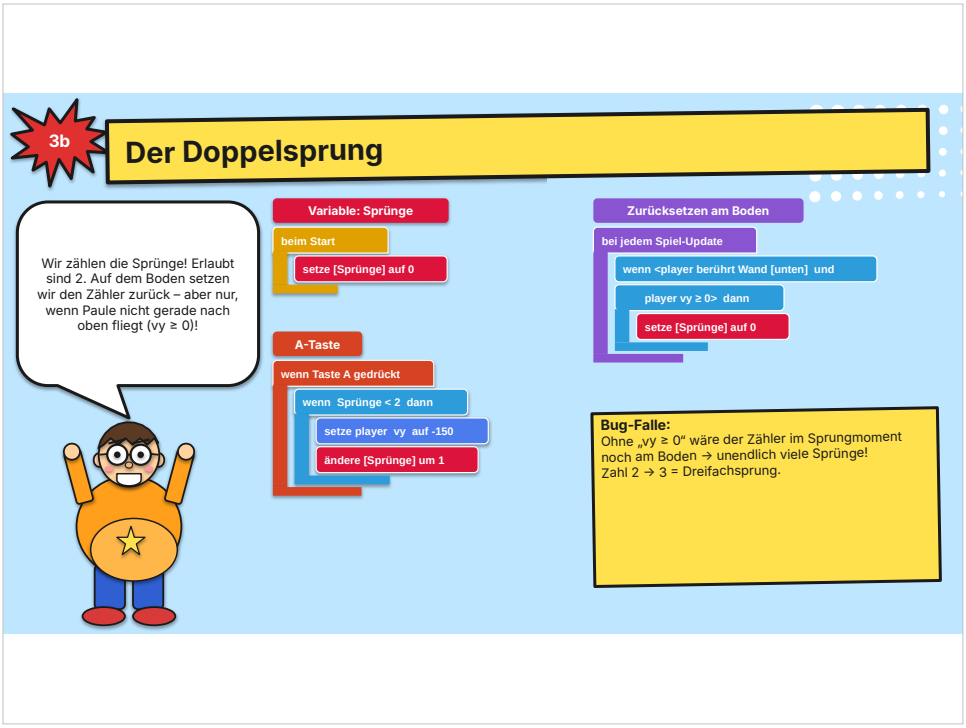
Neue SpriteKinds (Sprites > SpriteKind bearbeiten): Player, Enemy, Food (Münzen), Boss.
JS: namespace SpriteKind { export const Boss = SpriteKind.create() }
Platzhalter-Kacheln werden zur Laufzeit per Schleife ersetzt (Kapitel 5, 6, 7).



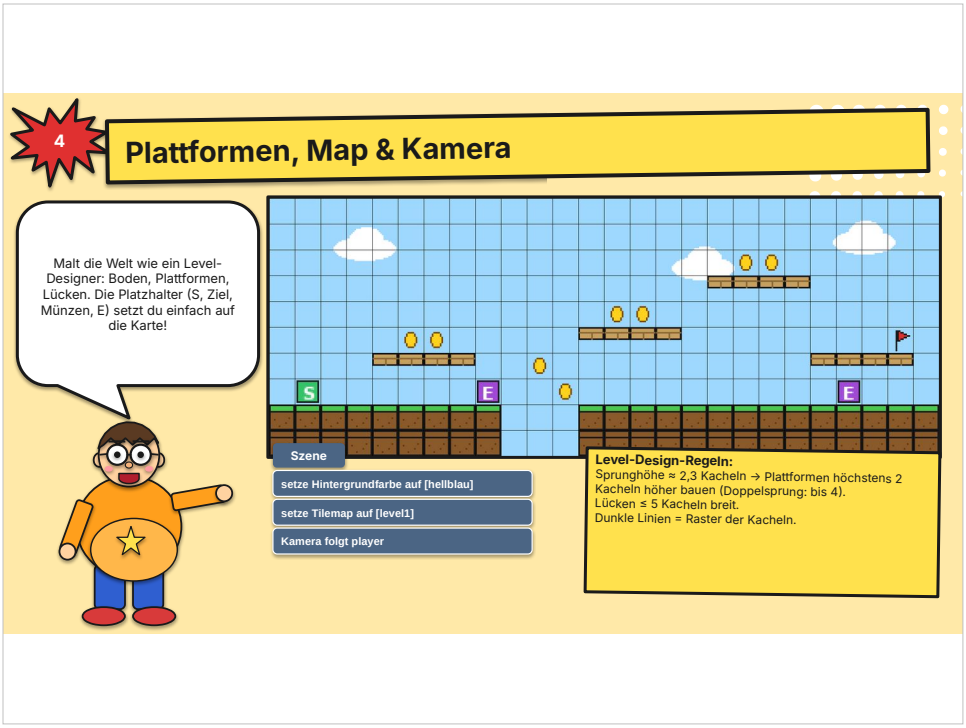

```
let player = sprites.create(assets.image`player`, SpriteKind.Player)
controller.moveSprite(player, 100, 0)
player.ay = 300
scene.cameraFollowSprite(player)
info.setLife(3)
```



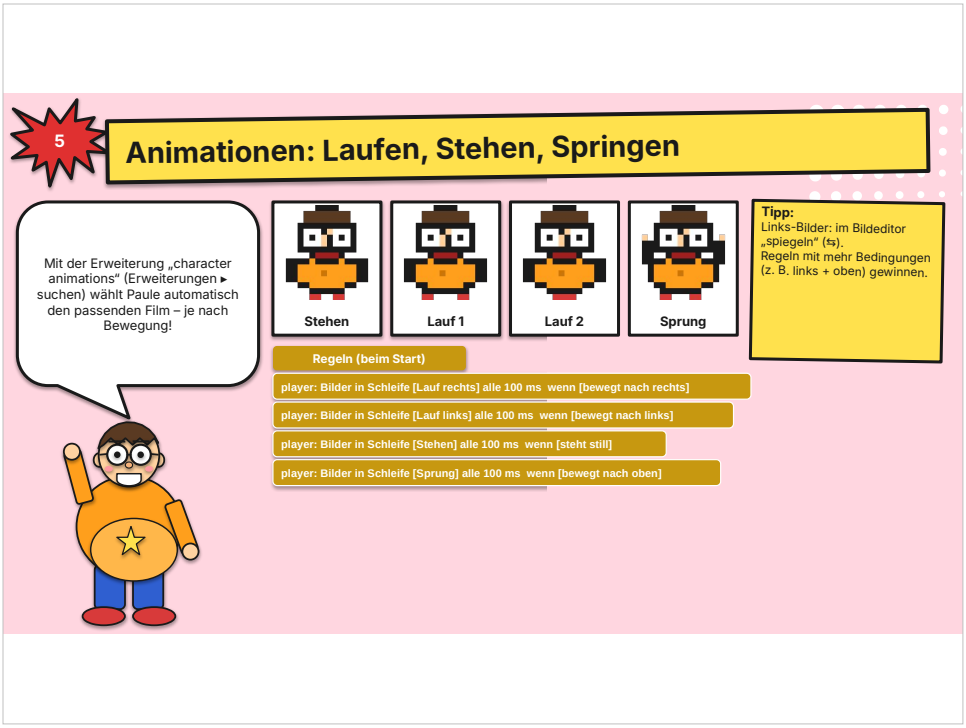

```
controller.A.onEvent(ControllerButtonEvent.
Pressed, function () {
    if
(player.isHittingTile(CollisionDirection.Bottom
)) {
        player.vy = -150
    }
})
```




```
let jumps = 0
controller.A.onEvent(ControllerButtonEvent.
Pressed, function () {
    if (jumps < 2) {
        player.vy = -150
        jumps += 1
    }
})
game.onUpdate(function () {
    if
(player.isHittingTile(CollisionDirection.Bottom
) && player.vy >= 0) {
        jumps = 0
    }
})
```

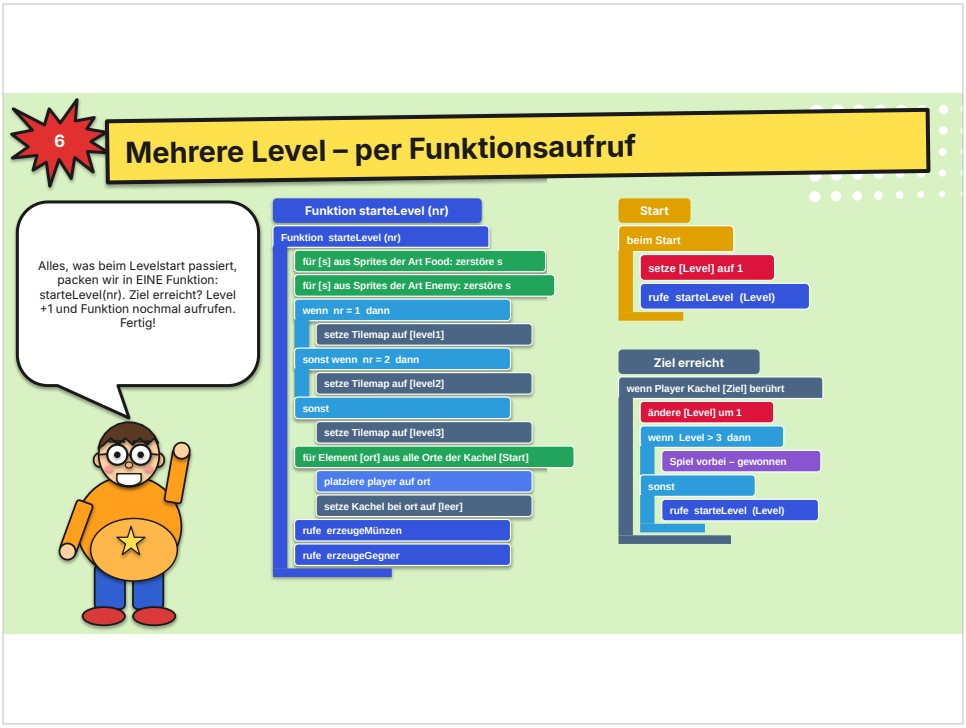



```
scene.setBackgroundColor(9)
tiles.setCurrentTilemap(tilemap`level1`)
scene.cameraFollowSprite(player)
```

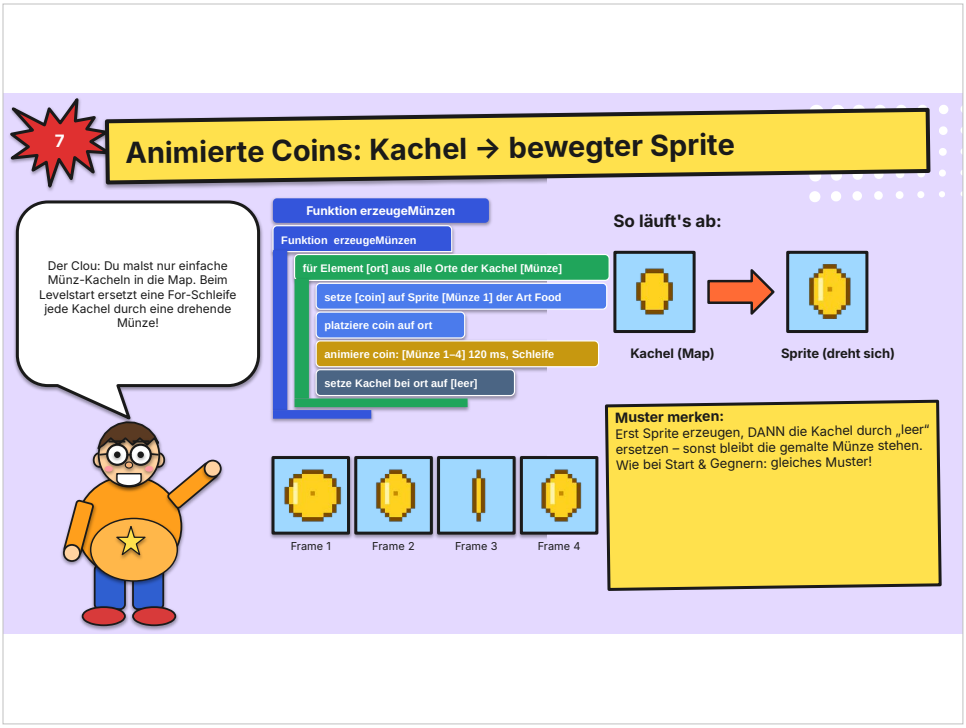


Erweiterung: character animations (Erweiterungen > 'character animations').

```
characterAnimations.loopFrames(player, assets.animation`walkRight`, 100, characterAnimations.rule(Predicate.MovingRight))
characterAnimations.loopFrames(player, assets.animation`walkLeft`, 100, characterAnimations.rule(Predicate.MovingLeft))
characterAnimations.loopFrames(player, [assets.image`idle`], 100, characterAnimations.rule(Predicate.NotMoving))
characterAnimations.loopFrames(player, [assets.image`jump`], 100,
```




```
let level = 0
function startLevel (nr: number) {
    for (let s of
sprites.allOfKind(SpriteKind.Food))
{ s.destroy() }
    for (let s of
sprites.allOfKind(SpriteKind.Enemy))
{ s.destroy() }
    if (nr == 1)
{ tiles.setCurrentTilemap(tilemap`level1`) }
    else if (nr == 2)
{ tiles.setCurrentTilemap(tilemap`level2`) }
    else
{ tiles.setCurrentTilemap(tilemap`level3`) }
    for (let ort of
tiles.getTilesByType(assets.tile`spawn`)) {
```



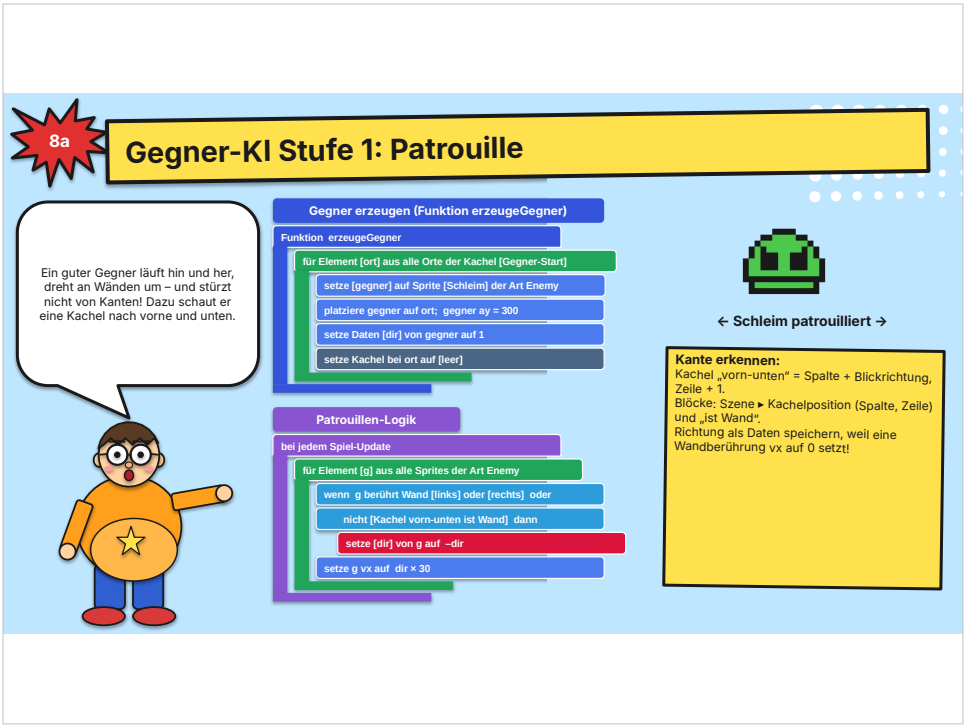

```
function spawnCoins () {
    for (let ort of
tiles.getTilesByType(assets.tile`coinTile`)) {
        let coin =
sprites.create(assets.image`coin0`,
SpriteKind.Food)
        tiles.placeOnTile(coin, ort)
        animation.runImageAnimation(coin,
assets.animation`coinSpin`, 120, true)
        tiles.setTileAt(ort,
assets.tile`transparency16`)
    }
}
```

Schweben (optional): game.onUpdate -> für jede Münze coin.y = startY + 2 * Math.sin(game.runtime() / 200)

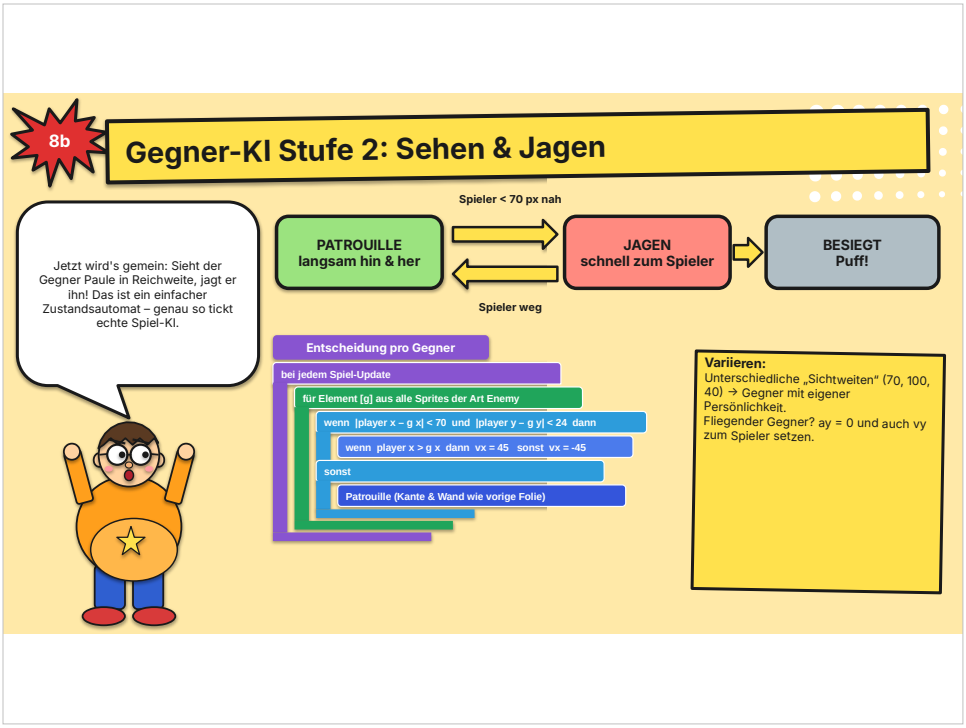



```
sprites.onOverlap(SpriteKind.Player, SpriteKind.Food, function (sprite, otherSprite) {
    info.changeScoreBy(1)
    otherSprite.destroy(effects.spray, 100)
    music.baDing.play()
})
game.onUpdate(function () {
    if (player.y > 320) {
        info.changeLifeBy(-1)
        startLevel(level)
    }
})
```

320 = Kartenhöhe in Pixel (Zeilen × 16) + etwas Reserve.



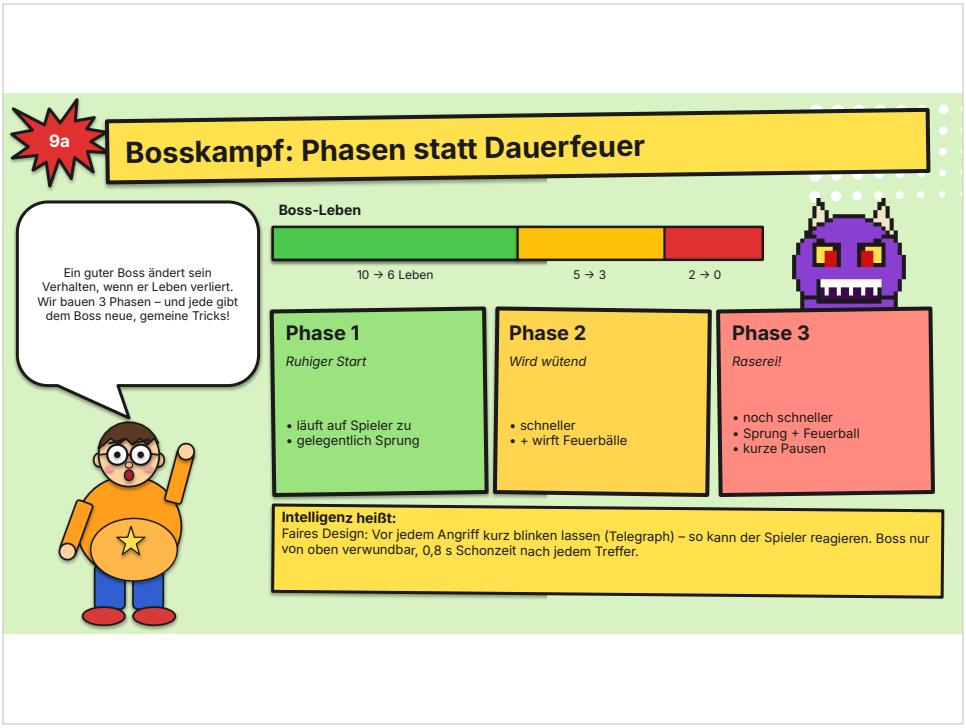

```
function spawnEnemies () {
    for (let ort of
tiles.getTilesByType(assets.tile`enemySpaw
n`)) {
        let enemy =
sprites.create(assets.image`slime`,
SpriteKind.Enemy)
        tiles.placeOnTile(enemy, ort)
        enemy.ay = 300
        sprites.setDataNumber(enemy, "dir", 1)
        tiles.setTileAt(ort,
assets.tile`transparency16`)
    }
}
game.onUpdate(function () {
    for (let enemy of
```



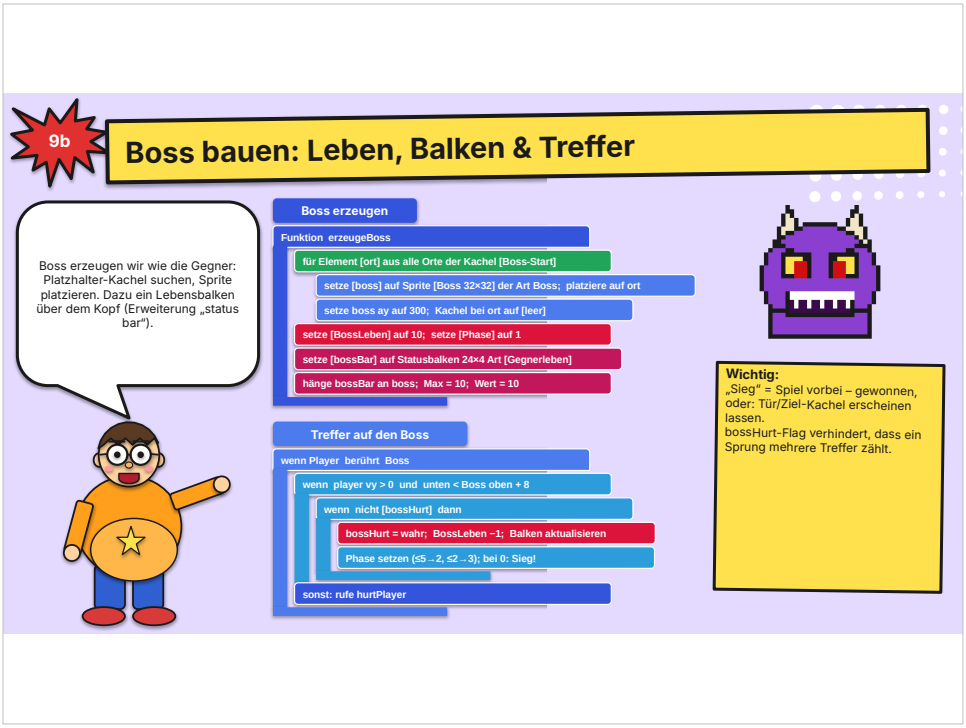

```
game.onUpdate(function () {
    for (let enemy of
sprites.allOfKind(SpriteKind.Enemy)) {
        if (Math.abs(player.x - enemy.x) < 70
&& Math.abs(player.y - enemy.y) < 24) {
            // JAGEN
            enemy.vx = player.x > enemy.x ? 45 :
-45
        } else {
            // PATROUILLE (wie auf der vorigen
Folie)
        }
    }
})
```



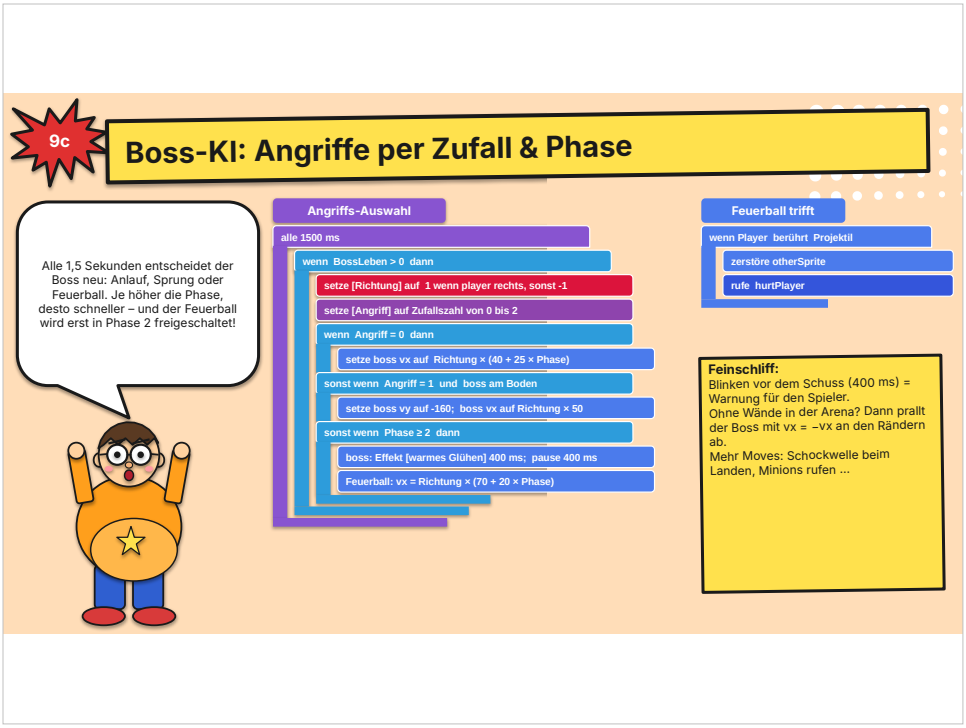

```
let invincible = false
function hurtPlayer () {
    if (!(invincible)) {
        invincible = true
        info.changeLifeBy(-1)
        player.vy = -120
        player.startEffect(effects.fire, 500)
        control.runInParallel(function () {
            pause(1000)
            invincible = false
        })
    }
}
sprites.onOverlap(SpriteKind.Player,
SpriteKind.Enemy, function (sprite,
otherSprite) {
```



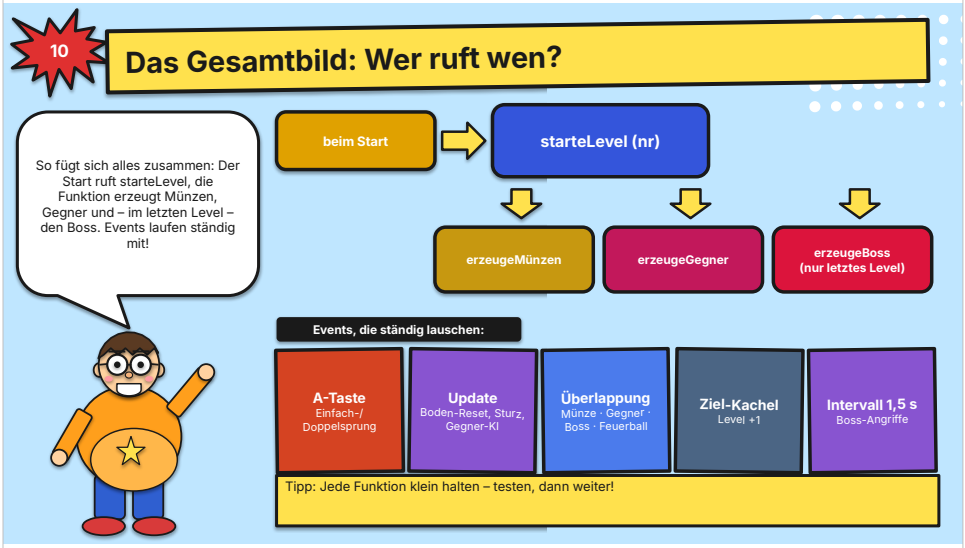
Phasen aus den Lebenspunkten ableiten: bossHp > 5 -> Phase 1; <= 5 -> Phase 2; <= 2 -> Phase 3.
Statusbalken: Erweiterung 'Status Bar' (Erweiterungen > 'status bar').



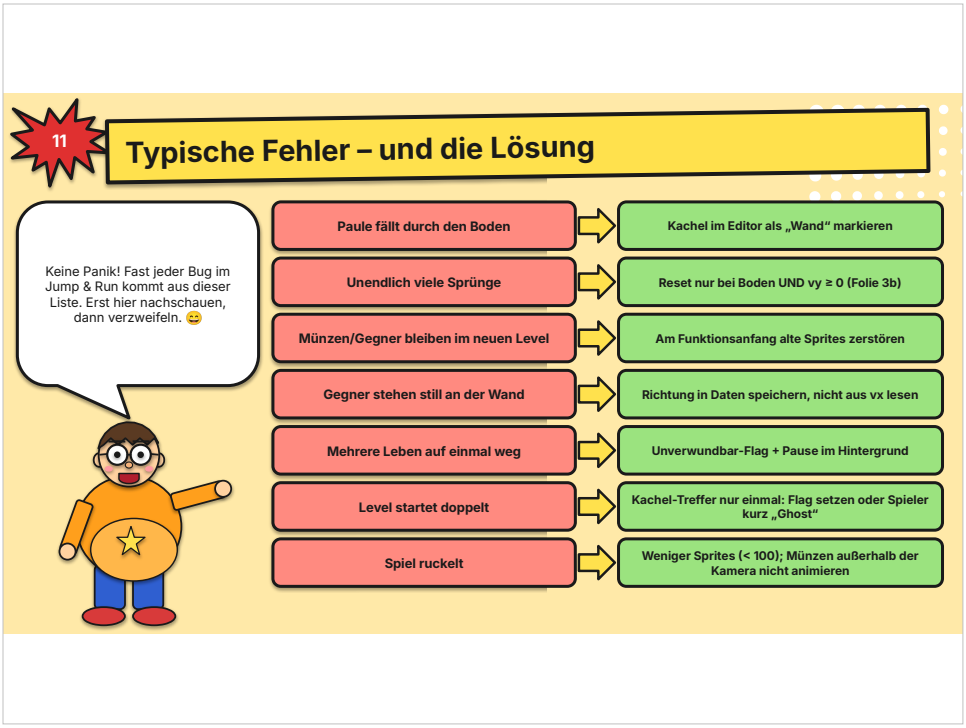

```
let boss: Sprite = null
let bossBar: StatusBarSprite = null
let bossHp = 0
let phase = 1
let bossHurt = false
function spawnBoss () {
    for (let ort of
tiles.getTilesByType(assets.tile`bossSpawn`)
) {
        boss =
sprites.create(assets.image`boss`,
SpriteKind.Boss)
        tiles.placeOnTile(boss, ort)
        boss.ay = 300
        tiles.setTileAt(ort,
assets.tile`transparency16`)
```




```
namespace SpriteKind { export const Boss =
SpriteKind.create() }
game.onUpdateInterval(1500, function () {
    if (bossHp > 0) {
        let dir = player.x > boss.x ? 1 : -1
        let attack = randint(0, 2)
        if (attack == 0) {
            boss.vx = dir * (40 + 25 * phase)
        } else if (attack == 1 &&
boss.isHittingTile(CollisionDirection.Bottom))
{
            boss.vy = -160
            boss.vx = dir * 50
        } else if (phase >= 2) {
            boss.startEffect(effects.warmRadial,
400)
```



Struktur:
beim Start -> level = 1; startLevel(level)
startLevel -> Tilemap laden, Spieler setzen, spawnCoins(), spawnEnemies(), (nur Level 3) spawnBoss()
Events: A-Taste, onUpdate (Boden, Sturz, Gegner-KI), Überlappungen (Münze, Gegner, Boss, Feuerball, Ziel).



Fehlersuche-Tabelle.



Ideen:
- Coyote Time: nach dem Verlassen der Kante noch 0,1 s springen dürfen
- Wandsprung: an Wand vy = -150 und vx wegdrücken
- Checkpoint-Kachel
- Bewegliche Plattform: Sprite mit vx, Spieler fährt mit (player.x += plat.vx*dt)
- Schlüssel & Tür, Power-Ups (Pilz = Doppelsprung nur kurz)


VIEL SPASS
BEIM HÜPFEN!
Und denk dran: Fehler sind nur Level, die du noch nicht geschafft hast. Bis zum nächsten Boss!

Abschluss.